

Machine Learning 2024-2045
Homework 3 - Group 127

I. - Pen-and-Paper
 (ist1106900, ist1106937)

Pen - and - paper

①

D	y_1	y_2	y_{num}	y_{class}
n_1	1	1	1.25	B
n_2	1	3	7	A
n_3	3	2	2.7	C
n_4	3	3	3.2	A
n_5	2	4	5.5	B

Transformed input matrix:

$$\Phi = \begin{bmatrix} 1 \\ 3 \\ 6 \\ 9 \\ 8 \end{bmatrix}$$

OLS Design matrix: $X = \begin{bmatrix} 1 & 1 \\ 1 & 3 \\ 1 & 6 \\ 1 & 9 \\ 1 & 8 \end{bmatrix}$

$X^T \rightarrow$ transpose of X

$(X^T X)^{-1} \rightarrow$ inverse of $X^T X$

$$w = \begin{bmatrix} 3.31593 \\ 0.11372 \end{bmatrix}$$

OLS regression model:

$$\hat{y}_{num} = 3.31593 + 0.11372 \times \phi(y_1, y_2)$$

Final regression model:

$$\hat{y}_{num} = 3.31593 + 0.11372 \cdot (y_1 \times y_2)$$

$$\phi(y_1, y_2) = y_1 \times y_2$$

Transforming input space

$$\phi(n_1) = \phi(1, 1) = 1 \times 1 = 1$$

$$\phi(n_2) = \phi(1, 3) = 1 \times 3 = 3$$

$$\phi(n_3) = \phi(3, 2) = 3 \times 2 = 6$$

$$\phi(n_4) = \phi(3, 3) = 3 \times 3 = 9$$

$$\phi(n_5) = \phi(2, 4) = 2 \times 4 = 8$$

Target values: $Y = \begin{bmatrix} 1.25 \\ 7 \\ 2.7 \\ 3.2 \\ 5.5 \end{bmatrix}$

OLS closed form solution: $w = (X^T X)^{-1} X^T Y$

To calculate the resulting vector, we used a small python code ~~and~~ and the `linalg.pinv` function (from the hint).

Here is the code:

`import numpy as np`

`X = np.array([[1, 1], [1, 3], [1, 6], [1, 9], [1, 8]])`

`Y = np.array([1.25, 7.00, 2.70, 3.20, 5.50])`

`w = np.linalg.pinv(X.T @ X) @ X.T @ Y`

```
import numpy as np

# Design matrix X
X = np.array([[1, 1],
               [1, 3],
               [1, 6],
               [1, 9],
               [1, 8]])

# Target vector Y (y_num)
Y = np.array([1.25, 7.00, 2.70, 3.20, 5.50])

# Compute the OLS solution
w = np.linalg.pinv(X.T @ X) @ X.T @ Y
```

Pen-and-paper

② Ridge Regression

$$J(\hat{w}) = \sum (y_i - \hat{y}_i)^2 + \lambda \sum w_i^2$$

$\lambda \rightarrow$ penalty factor
 $w_i \rightarrow$ weights

The larger the λ , the stronger the penalty

From the previous exercise: $\Phi = \begin{bmatrix} 1 \\ 3 \\ 6 \\ 9 \\ 8 \end{bmatrix}$

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 3 \\ 1 & 6 \\ 1 & 9 \\ 1 & 8 \end{bmatrix}$$

$$Y = \begin{bmatrix} 1.25 \\ 7.00 \\ 2.70 \\ 3.20 \\ 5.50 \end{bmatrix}$$

Ridge formula:

$$\hat{w}_{\text{ridge}} = (X^T X + \lambda I)^{-1} \cdot X^T \cdot Y$$

we used the same code as the previous exercise, but ~~added~~ added these lines:

Using the code, and $\lambda = 1$,

we got the intercept bias = 1.81809

slope = 0.32376

• `lambda = 1` # beginning of code

• `I = np.eye(X.shape[1])` # after the arrays

• `w_ridge = np.linalg.pinv(X.T @ X + lambda * I) @ X.T @ Y`

$$\hat{y}_{\text{ridge num}} = 1.81809 + 0.32376 \times \varphi(y_1, y_2)$$

When penalizing the intercept in Ridge regression, the intercept drops from 3.31543 to 1.81809, while the slope increases from 0.11342 to 0.32376, as the model compensates for the smaller intercept by assigning more weight to the feature coefficient.

Regularization in Ridge regression significantly reduces the intercept and increases the slope, redistributing the model's complexity from the intercept to the feature coefficients.

```
import numpy as np

# Define the Lambda (penalty) value
lambda_ = 1

# Design matrix X (with intercept) and target vector Y
X = np.array([[1, 1],
              [1, 3],
              [1, 6],
              [1, 9],
              [1, 8]])

Y = np.array([1.25, 7.00, 2.70, 3.20, 5.50])

# Identity matrix for regularization
I = np.eye(X.shape[1])

# Ridge regression closed-form solution
w_ridge = np.linalg.pinv(X.T @ X + lambda_ * I) @ X.T @ Y
w_ridge
```


Pen-and-Paper

② Root Mean Square Error (RMSE) : $\sqrt{\frac{1}{n} \sum_{i=1}^n (y_{true} - y_{pred})^2}$

$\cdot n_6 = (2, 2), y_6 = 0.7, \phi(n_6) = 2 \times 2 = 4$

$\cdot n_7 = (1, 2), y_7 = 1.1, \phi(n_7) = 1 \times 2 = 2$

$\cdot n_8 = (5, 1), y_8 = 2.2, \phi(n_8) = 5 \times 1 = 5$

Predictions For Training Data (OLS) : $X = \begin{bmatrix} 1 \\ 3 \\ 6 \\ 9 \\ 8 \end{bmatrix}$, from exercise 1
 $Y = [1.25; 7; 2.7; 3.2; 5.5]$

$\hat{y}_{OLS} = 3.31543 + 0.11372 \times X_i$

$X_1 = 1; \hat{y} = 3.31543 + 0.11372 \times 1 = 3.42915$

$X_2 = 3; \hat{y} = 3.31543 + 0.11372 \times 3 = 3.65109$

$X_3 = 6; \hat{y} = \text{" " " } \times 6 = 4.01625$

$X_4 = 9; \hat{y} = \text{" " " } \times 9 = 4.37541$

$X_5 = 8; \hat{y} = \text{" " " } \times 8 = 4.26169$

Predictions for training Data (Ridge) :

$\hat{y}_{ridge} = 1.81809 + 0.32376 \times X_i$, from exercise 2

$X_1 = 1; \hat{y} = 1.81809 + 0.32376 \times 1 = 2.14185$

$X_2 = 3; \hat{y} = \text{" " " } \times 3 = 2.78937$

$X_3 = 6; \hat{y} = \text{" " " } \times 6 = 3.76065$

$X_4 = 9; \hat{y} = \text{" " " } \times 9 = 4.73193$

$X_5 = 8; \hat{y} = \text{" " " } \times 8 = 4.40817$

OLS Training RMSE

Errors = $[y_{true} - y_{pred}] = [1.25 - 3.42915; 7 - 3.65109; 2.7 - 4.01625; 3.2 - 4.37541; 5.5 - 4.26169] = [-2.17465; 3.34291; -1.31625; -1.17541; 1.23831]$

Errors² = $[4.75089; 11.17608; 1.73252; 1.38154; 1.53342]$

$\sum \text{Errors}^2 = 20.5745$

$RMSE_{training}^{OLS} = \sqrt{\frac{20.5745}{5}} = 2.02650$

③ Cont.

Ridge Training RMSE:

$$\begin{aligned} \text{Errors} &= [1.15 - 4.14185; 7 - 2.78937; 2.7 - 3.76065; 3.2 - 4.73743; 5.5 - 4.40817] \\ &= [-0.84185; 4.21063; -1.06065; -1.53743; 1.09183] \end{aligned}$$

$$\text{Errors}^2 = [0.71494; 17.73339; 1.125; 2.36481; 1.1911]$$

$$\sum \text{Errors}^2 = 23.1927$$

$$\text{RMSE}_{\text{training}}^{\text{ridge}} = \sqrt{\frac{23.1927}{5}} = 2.15354$$

Predictions for test Data (OLS):

$$n_6; \hat{y} = 3.37593 + 0.11372 \times 4 = 3.77081$$

$$n_7; \hat{y} = " \quad " \quad " \quad \times 2 = 3.54337$$

$$n_8; \hat{y} = " \quad " \quad " \quad \times 5 = 3.88753$$

Predictions for test Data (Ridge)

$$n_6; \hat{y} = 1.87809 + 0.32376 \times 4 = 3.11313$$

$$n_7; \hat{y} = " \quad " \quad " \quad \times 2 = 2.46561$$

$$n_8; \hat{y} = " \quad " \quad " \quad \times 5 = 3.43789$$

OLS Test RMSE

$$\text{Errors} = [0.7 - 3.77081; 1.1 - 3.54337; 2.2 - 3.88753] = [-3.07081; -2.44337; -1.68753]$$

$$\text{Errors}^2 = [9.42988; 5.96805; 2.83864]$$

$$\sum \text{Errors}^2 = 18.23657$$

$$\text{RMSE}_{\text{test}}^{\text{OLS}} = \sqrt{\frac{18.23657}{3}} = 2.46560$$

Ridge Test RMSE

$$\text{Errors} = [0.7 - 3.11313; 1.1 - 2.46561; 2.2 - 3.43789] = [-2.41313; -1.36561; -1.23789]$$

$$\text{Errors}^2 = [5.8252; 1.86489; 1.53238]$$

$$\sum \text{Errors}^2 = 9.22247$$

$$\text{RMSE}_{\text{test}}^{\text{ridge}} = \sqrt{\frac{9.22247}{3}} = 1.75290$$

Conclusion

- Ridge regression has a significantly lower RMSE on the test set compared to OLS. This aligns with the expected behaviour, Ridge regularization improves generalization by penalizing the coefficients, which helps the model perform better on unseen data.

③ Cont.

Pen-and-Paper

OLS performs ^{slightly} better on the training set than Ridge, which is expected because OLS minimizes the training error without any regularization. Ridge, however, sacrifices some training performance to prevent overfitting, which is reflected in the slightly higher error in training.

④ Steps:

1. Forward Pass
2. Compute loss
3. Backward Pass
4. Update Weights and Biases

• Input: $x_1(1,1)$

• First layer computation: $z^{(1)} = W^{(1)} \cdot x_1 + b^{(1)}$

$$z^{(1)} = \begin{bmatrix} 0.1 & 0.1 \\ 0.1 & 0.2 \\ 0.2 & 0.1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0 \\ 0.1 \end{bmatrix} \quad (=)$$

$$\Rightarrow z^{(1)} = \begin{bmatrix} 0.1 \times 1 + 0.1 \times 1 \\ 0.1 \times 1 + 0.2 \times 1 \\ 0.2 \times 1 + 0.1 \times 1 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0 \\ 0.1 \end{bmatrix} \Rightarrow z^{(1)} = \begin{bmatrix} 0.2 \\ 0.3 \\ 0.3 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0 \\ 0.1 \end{bmatrix} (=)$$

$$\Rightarrow z^{(1)} = \begin{bmatrix} 0.3 \\ 0.3 \\ 0.4 \end{bmatrix}$$

• Second Layer Computation: $z^{(2)} = W^{(2)} \cdot z^{(1)} + b^{(2)}$

$$z^{(2)} = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0.3 \\ 0.3 \\ 0.4 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.3 \times 1 + 2 \cdot 0.3 + 2 \cdot 0.3 \\ 1 \times 0.3 + 2 \cdot 0.3 + 1 \cdot 0.3 \\ 1 \times 0.4 + 1 \cdot 0.3 + 1 \cdot 0.4 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1.7 \\ 1.3 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2.7 \\ 2.3 \\ 2 \end{bmatrix}$$

• Applying Softmax: $\text{Softmax}(z_c^{(2)}) = \frac{e^{z_c^{(2)}}}{\sum_{i=1}^K e^{z_i^{(2)}}}$

$$\sum_{i=1}^3 e^{z_i^{(2)}} = e^{2.7} + e^{2.3} + e^2 = 32.2429$$

$$e^{z^{(2)}} = \begin{bmatrix} e^{2.7} \\ e^{2.3} \\ e^2 \end{bmatrix} = \begin{bmatrix} 14.8797 \\ 9.9742 \\ 7.3891 \end{bmatrix}$$

Predicted probabilities for classes A, B, C:

$$\hat{y}_A = \frac{14.8797}{32.2429} = 0.4619 \quad \hat{y}_B = \frac{9.9742}{32.2429} = 0.3093 \quad \hat{y}_C = \frac{7.3891}{32.2429} = 0.2297$$

Cross-Entropy Loss

for x_1 , the true label is class A, so target vector t is: $t = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$

$$L = - \sum_{c=1}^K t_c \cdot \log(\hat{y}_c) \quad , \text{ this simplifies to: } L = -\log(\hat{y}_A) = -\log(0.4619) = 0.7731$$

④ cont.

Gradient at output layer

$$\frac{\partial L}{\partial z_i^{(1)}} = y_i - t_i$$

For each class:

$$\frac{\partial L}{\partial z_A^{(1)}} = 0,4615 - 1 = -0,5385$$

$$\frac{\partial L}{\partial z_B^{(1)}} = 0,3093 - 0 = 0,3093$$

$$\frac{\partial L}{\partial z_C^{(1)}} = 0,2291 - 0 = 0,2291$$

Gradient for second layer weights and biases

weights: $\frac{\partial L}{\partial w_{ij}^{(2)}} = \frac{\partial L}{\partial z_i^{(2)}} \times z_j^{(1)}$

biases: $\frac{\partial L}{\partial b_i^{(2)}} = \frac{\partial L}{\partial z_i^{(2)}}$

$$\frac{\partial L}{\partial z^{(2)}} = \begin{bmatrix} -0,5385 \\ 0,3093 \\ 0,2291 \end{bmatrix}$$

$$z^{(1)} = \begin{bmatrix} 0,3 \\ 0,3 \\ 0,4 \end{bmatrix}$$

• For class A (output $z_A^{(2)}$):

$$\frac{\partial L}{\partial w_{A1}^{(2)}} = -0,5385 \times 0,3 = -0,16155$$

$$\frac{\partial L}{\partial w_{A2}^{(2)}} = -0,5385 \times 0,3 = -0,16155$$

$$\frac{\partial L}{\partial w_{A3}^{(2)}} = -0,5385 \times 0,4 = -0,2154$$

• For class B (output $z_B^{(2)}$):

$$\frac{\partial L}{\partial w_{B1}^{(2)}} = 0,3093 \times 0,3 = 0,09279$$

$$\frac{\partial L}{\partial w_{B2}^{(2)}} = 0,3093 \times 0,3 = 0,09279$$

$$\frac{\partial L}{\partial w_{B3}^{(2)}} = 0,3093 \times 0,4 = 0,12372$$

For class C (output $z_C^{(2)}$):

$$\frac{\partial L}{\partial w_{C1}^{(2)}} = 0,2291 \times 0,3 = 0,06873$$

$$\frac{\partial L}{\partial w_{C2}^{(2)}} = 0,2291 \times 0,3 = 0,06873$$

$$\frac{\partial L}{\partial w_{C3}^{(2)}} = 0,09164 = 0,2291 \times 0,4$$

Gradient Matrix for $W^{(2)}$:

$$\frac{\partial L}{\partial W^{(2)}} = \begin{bmatrix} -0,16155 & -0,16155 & -0,2154 \\ 0,09279 & 0,09279 & 0,12372 \\ 0,06873 & 0,06873 & 0,09164 \end{bmatrix}$$

Gradient Matrix for $b^{(2)}$: $\frac{\partial L}{\partial b^{(2)}} = \begin{bmatrix} -0,5385 \\ 0,3093 \\ 0,2291 \end{bmatrix}$

④ cont.

Backpropagation to the first layer

$$\frac{\partial L}{\partial z^{(2)}} = \begin{bmatrix} -0.5385 \\ 0.3093 \\ 0.2291 \end{bmatrix}$$

$$\frac{\partial L}{\partial z_j^{(1)}} = \sum_i \frac{\partial L}{\partial z_i^{(2)}} \cdot W_{ij}^{(2)}$$

$$z_1^{(1)}: \frac{\partial L}{\partial z_1^{(1)}} = (-0.5385 \times 1) + (0.3093 \times 1) + (0.2291 \times 1) = -0.0001$$

$$z_2^{(1)}: \frac{\partial L}{\partial z_2^{(1)}} = (-0.5385 \times 2) + (0.3093 \times 2) + (0.2291 \times 1) = -0.2293$$

$$z_3^{(1)}: \frac{\partial L}{\partial z_3^{(1)}} = (-0.5385 \times 2) + (0.3093 \times 1) + (0.2291 \times 1) = -0.5386$$

$$\frac{\partial L}{\partial z^{(1)}} = \begin{bmatrix} -0.0001 \\ -0.2293 \\ -0.5386 \end{bmatrix}$$

Gradients for the first layer weights $W^{(1)}$: $\frac{\partial L}{\partial W_{ij}^{(1)}} = \frac{\partial L}{\partial z_j^{(1)}} \times n_i$

$$n_i = (1, 1) = n_1 \quad \frac{\partial L}{\partial W^{(1)}} = \begin{bmatrix} -0.0001 \\ -0.2293 \\ -0.5386 \end{bmatrix} \times [1, 1] = \begin{bmatrix} -0.0001 & -0.0001 \\ -0.2293 & -0.2293 \\ -0.5386 & -0.5386 \end{bmatrix}$$

Gradients for the first layer biases $b^{(1)}$:

$$\frac{\partial L}{\partial b^{(1)}} = \begin{bmatrix} -0.0001 \\ -0.2293 \\ -0.5386 \end{bmatrix}$$

Update weights and biases:

SGD using ~~learning rate~~ $\eta = 0.1$

$$W_{new} = W_{old} - \eta \cdot \frac{\partial L}{\partial W}$$

$$b_{new} = b_{old} - \eta \cdot \frac{\partial L}{\partial b}$$

$$b_1^{(1)}: b_1^{(1)} = 1 - 0.1 \times (-0.5385) = 1.05385$$

$$b_2^{(1)} = 1 - 0.1 \times (0.3093) = 0.96907$$

$$b_3^{(1)} = 1 - 0.1 \times 0.2291 = 0.97709$$

$$b_{new}^{(1)} = \begin{bmatrix} 1.05385 \\ 0.96907 \\ 0.97709 \end{bmatrix}$$

$$W_{11}^{(2)}: W_{11}^{(2)} = 1 - 0.1 \times (-0.16155) = 1.016155$$

$$W_{12}^{(2)} = 2 - 0.1 \times (-0.16155) = 2.016155$$

$$W_{13}^{(2)} = 2 - 0.1 \times (-0.21551) = 2.021551$$

$$W_{21}^{(2)} = 0.990721$$

$$W_{22}^{(2)} = 1.990721$$

$$W_{23}^{(2)} = 0.987628$$

$$W_{31}^{(2)} = 0.993127$$

$$W_{32}^{(2)} = 0.993127$$

$$W_{33}^{(2)} = 0.990836$$

$$W_{new}^{(2)} = \begin{bmatrix} 1.016155 & 2.016155 & 2.021551 \\ 0.990721 & 1.990721 & 0.987628 \\ 0.993127 & 0.993127 & 0.990836 \end{bmatrix}$$

④ cont.

Pen-and-paper

for the $W_{new}^{(1)}$ and $b_{new}^{(1)}$ we followed the same steps for $W_{new}^{(2)}$ and $b_{new}^{(2)}$

$$W^{(1)}: W_{11}^{(1)} = 0,1 - 0,1 \times (-0,0001) = 0,10001$$

$$W_{12}^{(1)} = 0,1 - 0,1 \times (-0,0001) = 0,10001$$

$$W_{21}^{(1)} = 0,12243$$

$$W_{22}^{(1)} = 0,12243$$

$$W_{31}^{(1)} = 0,15386$$

$$W_{32}^{(1)} = 0,15386$$

$$W_{new}^{(1)} = \begin{bmatrix} 0,10001 & 0,10001 \\ 0,12243 & 0,12243 \\ 0,15386 & 0,15386 \end{bmatrix}$$

$$b_{new}^{(1)} = \begin{bmatrix} 0,10001 \\ 0,102243 \\ 0,15386 \end{bmatrix}$$